1

Collective Operation Patterns

Design Patterns Aimed at Handling Large Collections of Resources

Includes content adapted from *API Design Pattern*, 2021, Manning Publications && *RESTful Java Web Services, 3rd Ed.* , 2017, Packt Publishing

Collective Operation Patterns

1. **Copy and Move**
2. Batch Operations
3. Criteria-Based Deletion
4. Anonymous Writes
5. Pagination
6. Filtering
7. Import & export



Copy and Move

- Operations, covering both copying (duplication) and moving (changing a unique identifier or changing a parent) of resources in an API.
  - Since we cannot use the standard update method in order to move or copy a resource, we're left with the obvious next best choice: custom methods.
    - `/chatrooms/1/messages/2:move`
    - `/chatrooms/2:copy`
  - The high-level idea of how these copy-and-move custom methods might work is straightforward.
    - *The problem is in the details...*



Copy and Move

- Some important questions:
  - *How to deal with identifier?*
    - *User chooses new Id or system creates?*
    - *Is moving to different parents handle in the same ways as copying under same parent?*
  - *Deep copy?*
  - *Concurrent access.*
  - *What about references to the “old” resource?*



Copy and Move

- **Identifiers:**
  - Usually best to let system chose new identifier. **<u>However:</u>**
  - If renaming a resource with new identifier in the same position, then it makes sense it should be user-provided
    - `databases/prod - rename to -> databases/prod-old/`
  - When moving a resource to a new parent, it is often convenient to maintain the same id:
    - `chatroom/1/messages/abc`
      `- move to-> chatroom/2/messages/abc`



Copy and Move

- **Identifiers when copying:**
  - When copying (creating new resource), it should be the same as creating a new resource.
    - *Allow user to specify id only if it is also possible to do so when creating the resource.*

```
library/1/books/1:copy?destination=library.2
library/1/books/1:copy?destination=library.2&id=3
```



Copy and Move

- **Identifiers when copying (Example):**

  `library/1/books/1:copy?destination=library.2`
  - Copy to new parent and generate new id.

  `library/1/books/1:copy?destination=library.1`
  - Create duplicate under same parent

  `library/1/books/1:copy?destination=library.2&id=3`
  - Copy to new parent with specified id.
  - Generate 409 Conflict HTTP status if id is already taken.



Copy and Move

- **Identifiers when moving:**
- Two different scenarios. Either:
  - **Move to new parent (relocate) –** Requires specification of the target parent.
  - **Move under the same parent (renaming)** – *only if user-provided identifiers are allowed*. – requires specification of new identifier.

```
library/1/books/1:move?destinationParent=library.2
library/1/books/1:move?destinationId=33
```



Copy and Move

- **Child resources**
- Must child resources be copied?
  - In a nutshell, yes. They are there for a reason.
    - *Copying a chatroom will also copy all the messages.*

Resource type	Original identifier	New identifier
ChatRoom	chatRooms/old	chatRooms/new
Message	chatRooms/old/messages/2	chatRooms/new/messages/2
Message	chatRooms/old/messages/3	chatRooms/new/messages/3



Copy and Move

- **Related resources**
  - What about related resources?
    - E.g. A MessageReviewReport resource referencing a moved/copied message.
  - When a message is duplicated, should the referring *MessageReviewReport* also be duplicated?
  - When a message is moved, should the referring *MessageReviewReport* also be updated?
  - Its complicated. There is no single correct answer that works for all scenarios.



Copy and Move

- **Referential Integrity:**
  - Requires updating all referencing resources when moving resources. We'd like to ideally achieve this. But:
    - No easy way to keep track of all referring resources (as well as moved child resources).
    - Very resource intensive. Might require updating thousands of records...
    - And this is why moving resources around is actually discouraged...



Copy and Move

- **Duplicating Related Resources:**
  - Hard design decision, because it depends on circumstances.
    - *While it is not very sensible to duplicate a MessageReviewReport, it cannot just be ignored. It would make perhaps more sense to make the report able to refer to multiple messages instead.*
    - *If the user would create a copy of a chatroom, it would probably surprise him if the users were also duplicated.*
  - Bottom line, it depends on semantics and intended behavior.



Copy and Move

- **Atomicity:**
  - Copy and move should occur in the context of a transaction.
  - If any part of the operation fails (e.g. copying a sub-resource), then all operations should be aborted.
  - How to address concurrent accesses/modifications?
    - Use a database snapshot/transaction/lock the database for writing.
    - Or, ignore and hope for the best! Dangerous when moving resources, because it can undo previous updates...



Copy and Move

- **All this is very tricky, hard to deal with and might introduce inefficiency and subtle errors/inconsistencies or even lost and corrupted data.**
  - Copying and moving resources should be done only if strictly necessary!
  - Avoid these operations whenever possible!
  - Copying is less problematic than moving and much easier to handle.

Collective Operation Patterns

1. Copy and Move
2. **Batch Operations**
3. Criteria-Based Deletion
4. Anonymous Writes
5. Pagination
6. Filtering
7. Import & export



Batch Operations

- **Manipulate multiple resources with a single API call.**
- These aim to provide behaviour similar to standard methods, but can affect multiple resources with a single call.
  - Batch GET
  - Batch CREATE
  - Batch UPDATE
  - Batch DELETE
- We want these operations to be **atomic** in nature.



Batch Operations

- **Manipulate multiple resources with a single API call.**
- These aim to provide behaviour similar to standard methods, but can affect multiple resources with a single call.
  - Batch GET
  - Batch CREATE
  - Batch UPDATE
  - Batch DELETE
- We want these operations to be **atomic** in nature.
  - This is one of the main reasons these operations are provided.



Batch Operations

- **Result Ordering:**
- Results should preserve the order of the request.
  - *For example, when creating multiple resources, it is important for the client to be able to trace each created resource to each request, so that it may know the generated identifier for each resource.*



Batch Operations

Multiple Parents:

```
POST /chatRooms/-/messages:batchCreate HTTP/1.1
Content-Type: application/json

{
  "requests": [
    {
      "parent": "chatRooms/1",
      "resource": { ... }
    },
    {
      "parent": "chatRooms/2",
      "resource": { ... }
    }
  ]
}
```

We rely on a wild card hyphen character to indicate multiple parents, to be defined in the requests.

We define the actual parent in the list of requests to create the resources.



Batch Operations

Batch GET:

```
GET /chatrooms:batchGET?id=123&id=456

GET /chatrooms/1/messages:batchGET?id=m23&id=m446

GET /chatrooms/-/messages:batchGET?id=m23&id=m446
```

There should be an upper limit to the number of resources that can be retrieved.



Batch Operations

Batch DELETE:

```
POST /chatrooms:batchDelete?id=1&id=2

POST /chatrooms/1/messages:batchDelete?id=m23&id=m446

POST /chatrooms/-/messages:batchDelete?id=m23&id=m446
```

Batch Operations

Batch CREATE

```
POST /chatRooms/-/messages:batchCreate HTTP/1.1
Content-Type: application/json

{
  "requests": [
    {
      "parent": "chatRooms/1",
      "resource": { ... }
    },
    {
      "parent": "chatRooms/2",
      "resource": { ... }
    }
  ]
}
```

These must be consistent

Batch Operations

Batch UPDATE:

Global Fieldmask

```
POST /chatrooms/1/messages:batchUpdate?fieldMask=name&fieldMask=“date” HTTP/1.1
Content-Type: application/json

{
    “requests”: [
          {
           “fieldmask” : [“name”]
          “resource”   : {…}
           },
           {
          “resource”   : {…}
           }]
}
```

Specific Fieldmask

Collective Operation Patterns

1. Copy and Move
2. Batch Operations
3. **Criteria-Based Deletion**
4. Anonymous Writes
5. Pagination
6. Filtering
7. Import & export



Criteria-Based Deletion

- **While a batch delete can remove multiple resources, it requires knowing their ids.**
- Often, it is more useful to delete based on a specific criteria rather than explicitly providing resource ids.
  - Querying for the ids and batch deleting is not atomic.
- The Criteria-based deletion pattern should provide a mechanism allowing such operations to be conducted atomically and safely.

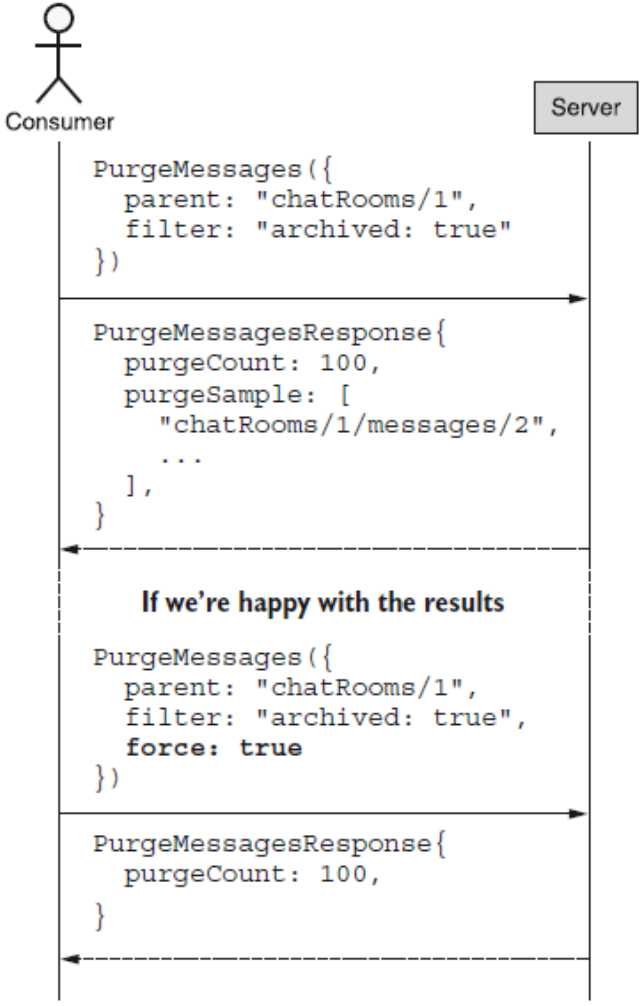


Criteria-Based Deletion

- **Although this functionality is useful, it is dangerous.**
- Users can, and will, make errors.
- Because of that, safeguards will be provided to reduce the possibility of catastrophic results:
  - A `force` Boolean flag that must be explicitly set on request.
  - A list previewing deleted resources and their count will be obtained in unforced requests.

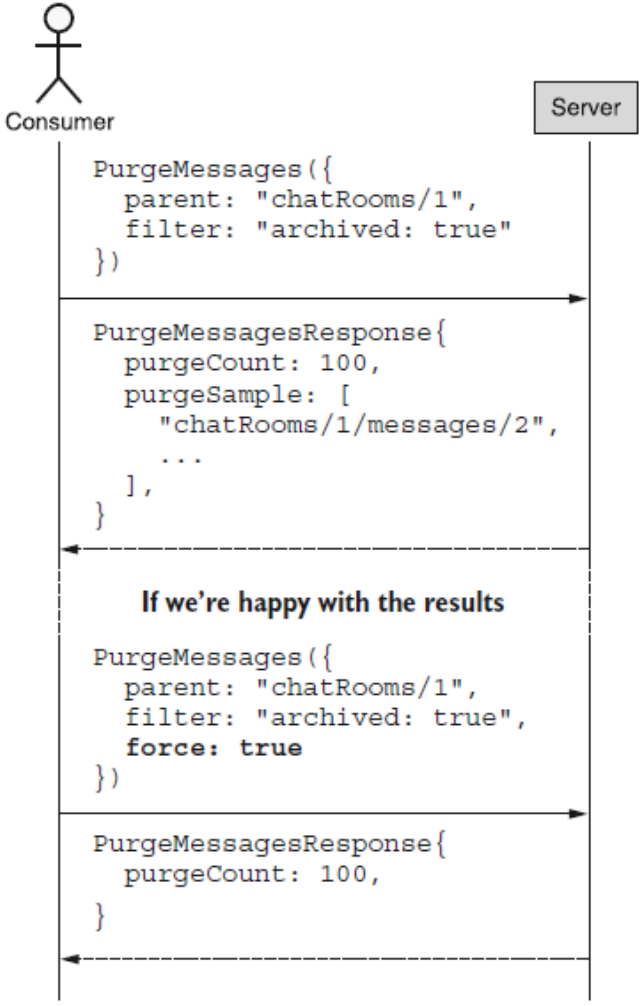
Criteria-Based Deletion

- **The user is expected to confirm the results before committing the decision by forcing the request.**
  - The filter should work similarly to the LIST standard method.
  - This is important for consistency, however the LIST default filter (recover all resources) which must be used for consistent behavior, can be problematic for deletion.



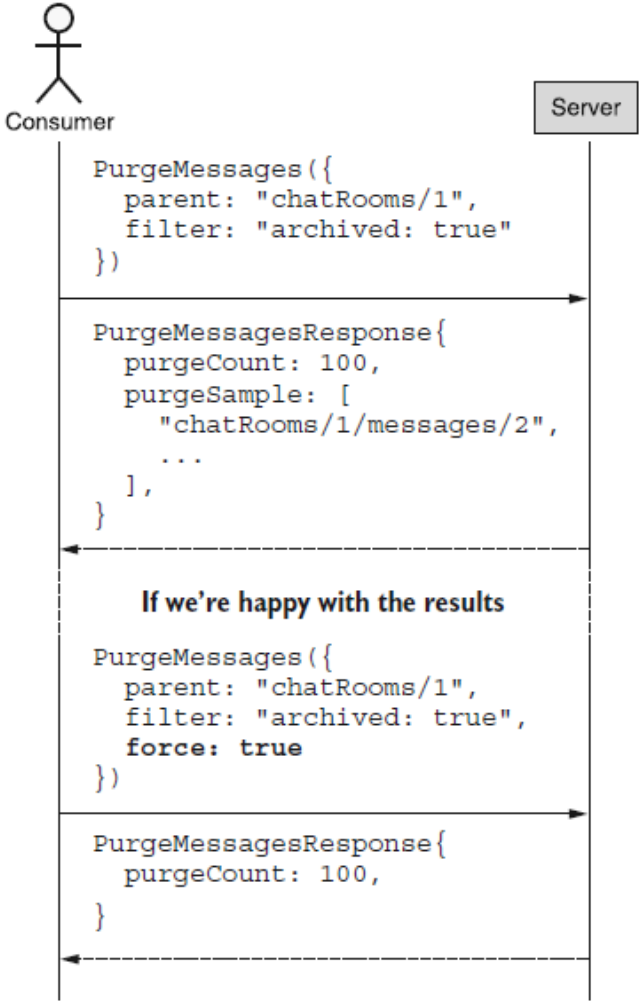
Criteria-Based Deletion

- **A partial sample of deleted resources is provided for inspection:**
  - Up to 100 (more or less) resources is a reasonable sample.
- What should be done if the resources changed between unforced and forced purges?



Criteria-Based Deletion

- **What if filter results are changed between unforced and forced purges?**
  - Delete also new resources that match filter?
  - Delete just those that were previewed?
- The latter is hard to enforce, might lead to bad results, and can be readily implemented if wanted by LISTing and Deleting separately.
- So, the former option is preferrable.



Collective Operation Patterns

1. Copy and Move
2. Batch Operations
3. Criteria-Based Deletion
4. **Anonymous Writes**
5. Pagination
6. Filtering
7. Import & export



Anonymous Writes

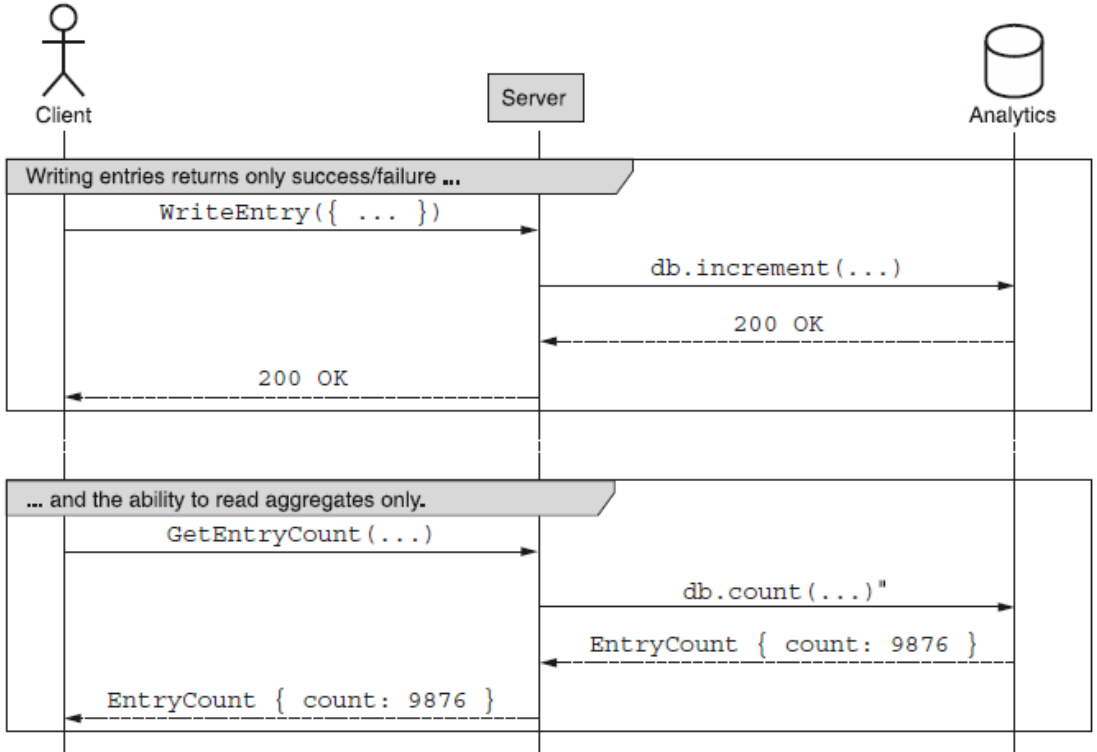
- **Sometimes, we want to store data that is not appropriately represented as a uniquely identifiable and addressable resource, due to granularity and variability.**
  - E.g. time series data representing the position of a car
- In this scenarios, the user is typically more interested in aggregated results rather than individual data points.



Anonymous Writes

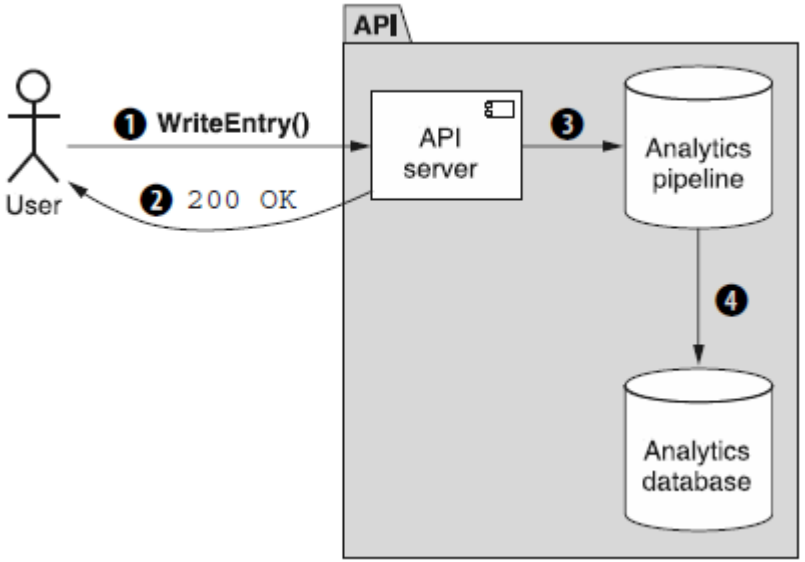
- **The anonymous write pattern provides a custom method `write` that allows users to provided such data.**
  - Using the standard create method for such purpose would make no sense as these handle only resources.
  - To distinguish clearly between resources and write-able information, the latter is referred to as `entry`.

Anonymous Writes



Anonymous Writes

- **The write method:**
  - Does not return a created resource.
  - Should target a collection of stats, e.g: Use `/chatRooms/1/statEntries:write` rather than `/chatRooms/1:writeStatEntry`
  - Due to the aggregate nature of entry information, write may return immediately before the entry info is propagated.



Collective Operation Patterns

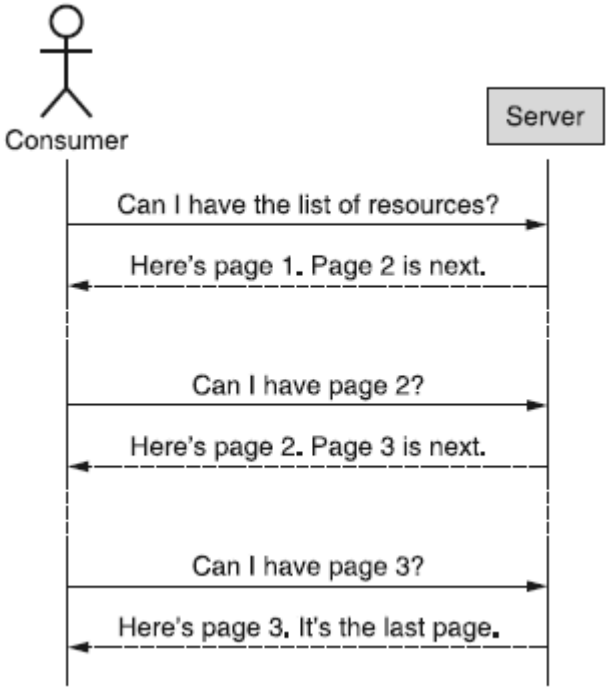
1. Copy and Move
2. Batch Operations
3. Criteria-Based Deletion
4. Anonymous Writes
5. **Pagination**
6. Filtering
7. Import & export



Pagination

- **The response to a user request may be unreasonably large.**
  - Many resources might be returned.
  - The information within a single resource might be too large.
- The Pagination pattern allows the response to be returned to the user in a piecewise manner with multiple pages of reasonable size.

Pagination

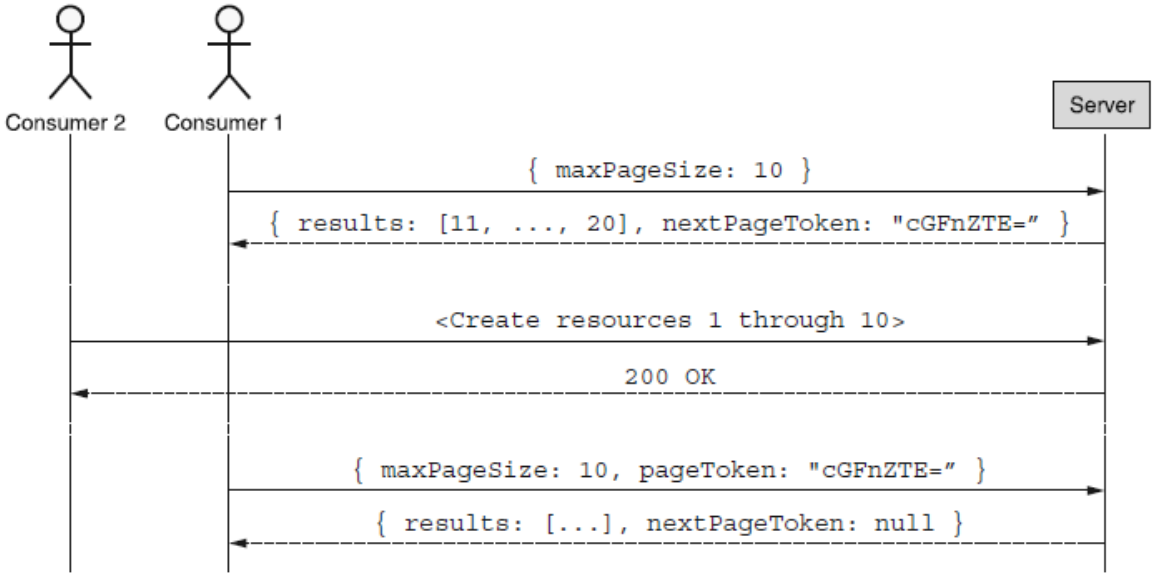

Consumer
Server
Can I have the list of resources?
Here's page 1. Page 2 is next.
Can I have page 2?
Here's page 2. Page 3 is next.
Can I have page 3?
Here's page 3. It's the last page.

Pagination


Consumer
Server
{ maxPageSize: 10 }
{ results: [...], nextPageToken: "cGFnZTE=" }
{ maxPageSize: 10, pageToken: "cGFnZTE=" }
{ results: [...], nextPageToken: "UGFHRTI=" }
{ maxPageSize: 10, pageToken: "UGFHRTI=" }
{ results: [...], nextPageToken: null }

Pagination

- A consumer might see duplicated results



Pagination

- How long should tokens remain valid_

